

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



DATA STRUCTURES AAT REPORT on

CODING CHALLENGES

Submitted by

NAME (1BM--CS---)

Under the Guidance of
Anusha S

Assistant Professor, BMSCE

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



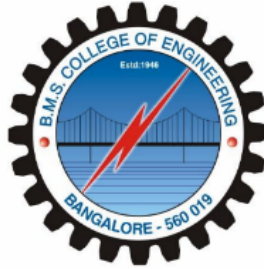
B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

August to December 2025

B.M.S. COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



DECLARATION

I, **NAME1 (1BM--CS---)** student of 3rd Semester F Section, B.E, Department of Computer Science and Engineering, B. M. S. College of Engineering, Bangalore, hereby declare that, this Coding Challenges for the course Data Structures (23CS3PCDST) has been carried out by me under the guidance of **Anusha S**, Assistant Professor, Department of CSE, B. M. S. College of Engineering, Bangalore during the academic semester August to December 2025.

I also declare that to the best of our knowledge and belief, the development reported here is not from part of any other report by any other students.

Signature

NAME1 (1BM--CS---)

Coding challenge 1:

Given a string *s*, remove duplicate letters so that every letter appears once and only once. You must make sure your result is the smallest in lexicographical order among all possible results.

Example 1:

Input: *s* = "bcabc"

Output: "abc"

Example 2:

Input: *s* = "cbacdcbc"

Output: "acdb"

Constraints:

- $1 \leq s.length \leq 10^4$
- *s* consists of lowercase English letters.

LeetCode Link: <https://leetcode.com/problems/remove-duplicate-letters/description/?envType=problem-list-v2&envId=stack>

Solution:

(Write the complete C/C++ code handwritten only)

Attach screenshot of the code with output on the LeetCode platform (any 2 screenshots)

Follow the same for all 5 coding challenges

Coding challenge 2:

Design your implementation of the circular queue. The circular queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle, and the last position is connected back to the first position to make a circle. It is also called "Ring Buffer".

One of the benefits of the circular queue is that we can make use of the spaces in front of the queue. In a normal queue, once the queue becomes full, we cannot insert the next element even if there is a space in front of the queue. But using the circular queue, we can use the space to store new values.

Implement the MyCircularQueue class:

- MyCircularQueue(k) Initializes the object with the size of the queue to be k.
- int Front() Gets the front item from the queue. If the queue is empty, return -1.
- int Rear() Gets the last item from the queue. If the queue is empty, return -1.
- boolean enQueue(int value) Inserts an element into the circular queue. Return true if the operation is successful.
- boolean deQueue() Deletes an element from the circular queue. Return true if the operation is successful.
- boolean isEmpty() Checks whether the circular queue is empty or not.
- boolean isFull() Checks whether the circular queue is full or not.

You must solve the problem without using the built-in queue data structure in your programming language.

LeetCode link: <https://leetcode.com/problems/design-circular-queue/description/?envType=problem-list-v2&envId=queue>

Solution:

Coding challenge 3:

On day 1, one person discovers a secret. You are given an integer delay, which means that each person will **share** the secret with a new person **every day**, starting from delay days after discovering the secret. You are also given an integer forget, which means that each person will **forget** the secret forget days after discovering it. A person **cannot** share the secret on the same day they forgot it, or on any day afterwards.

Given an integer n, return *the number of people who know the secret at the end of day n*. Since the answer may be very large, return it **modulo** $10^9 + 7$.

Example 1:

Input: n = 6, delay = 2, forget = 4

Output: 5

Explanation:

Day 1: Suppose the first person is named A. (1 person)

Day 2: A is the only person who knows the secret. (1 person)

Day 3: A shares the secret with a new person, B. (2 people)

Day 4: A shares the secret with a new person, C. (3 people)

Day 5: A forgets the secret, and B shares the secret with a new person, D. (3 people)

Day 6: B shares the secret with E, and C shares the secret with F. (5 people)

Example 2:

Input: n = 4, delay = 1, forget = 3

Output: 6

Explanation:

Day 1: The first person is named A. (1 person)

Day 2: A shares the secret with B. (2 people)

Day 3: A and B share the secret with 2 new people, C and D. (4 people)

Day 4: A forgets the secret. B, C, and D share the secret with 3 new people. (6 people)

Constraints:

- $2 \leq n \leq 1000$
- $1 \leq \text{delay} < \text{forget} \leq n$

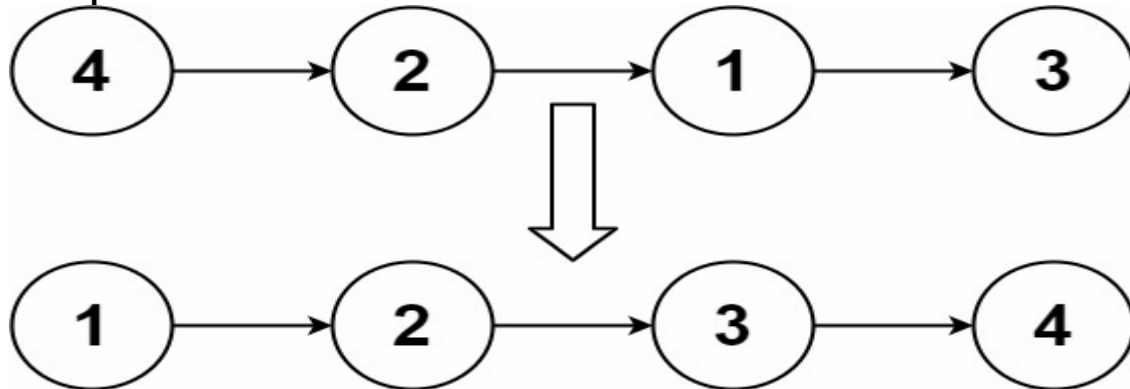
LeetCode link: <https://leetcode.com/problems/number-of-people-aware-of-a-secret/description/?envType=problem-list-v2&envId=queue>

Solution:

Coding challenge 4:

Given the head of a linked list, return *the list after sorting it in ascending order*.

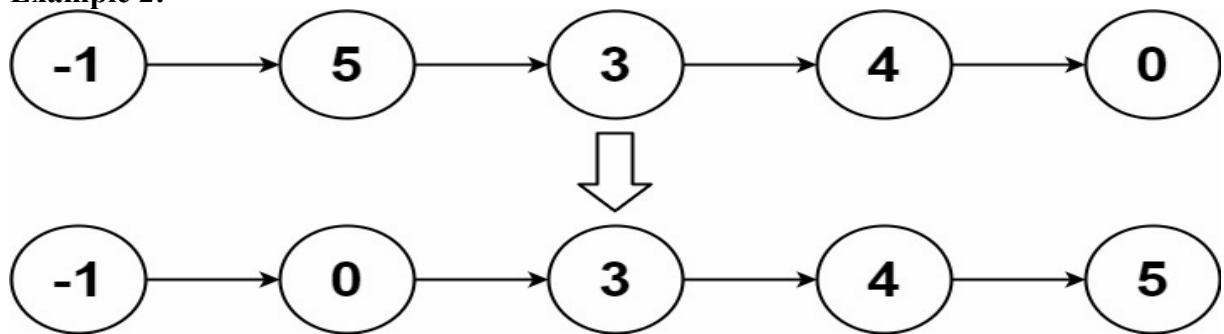
Example 1:



Input: head = [4,2,1,3]

Output: [1,2,3,4]

Example 2:



Input: head = [-1,5,3,4,0]

Output: [-1,0,3,4,5]

Example 3:

Input: head = []

Output: []

Constraints:

- The number of nodes in the list is in the range $[0, 5 * 10^4]$.
- $-10^5 \leq \text{Node.val} \leq 10^5$

LeetCode link: <https://leetcode.com/problems/sort-list/description/?envType=problem-list-v2&envId=linked-list>

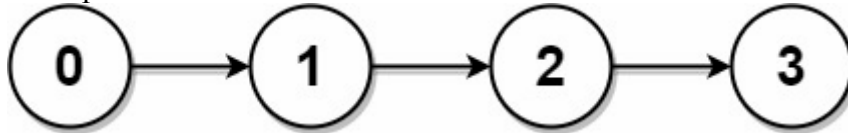
Solution:

Coding challenge 5:

You are given the head of a linked list containing unique integer values and an integer array `nums` that is a subset of the linked list values.

Return *the number of connected components in nums where two values are connected if they appear consecutively in the linked list*.

Example 1:

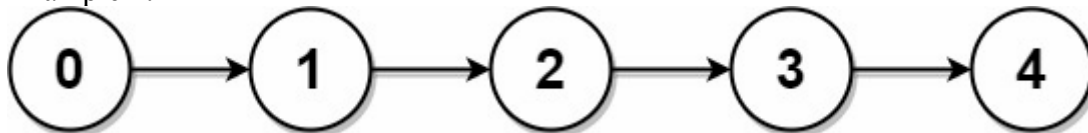


Input: `head = [0,1,2,3]`, `nums = [0,1,3]`

Output: 2

Explanation: 0 and 1 are connected, so `[0, 1]` and `[3]` are the two connected components.

Example 2:



Input: `head = [0,1,2,3,4]`, `nums = [0,3,1,4]`

Output: 2

Explanation: 0 and 1 are connected, 3 and 4 are connected, so `[0, 1]` and `[3, 4]` are the two connected components.

Constraints:

- The number of nodes in the linked list is `n`.
- $1 \leq n \leq 10^4$
- $0 \leq \text{Node.val} < n$
- All the values `Node.val` are unique.
- $1 \leq \text{nums.length} \leq n$
- $0 \leq \text{nums}[i] < n$
- All the values of `nums` are unique.

LeetCode link: <https://leetcode.com/problems/linked-list-components/description/?envType=problem-list-v2&envId=linked-list>

Solution:

